

COMP 3311

DATABASE MANAGEMENT SYSTEMS

LECTURE 23

TRANSACTION MANAGEMENT: CONCURRENCY CONTROL PROTOCOLS

LOCKING

Lock: a mechanism to control concurrent access to a data item.



- A data item Q can be locked in one of two modes:
 1. **shared-mode (shared lock)**
 - Can only read Q ; transaction requests with a **lock-s**(Q) instruction.
 2. **exclusive-mode (exclusive lock)**
 - Can both read and write Q ; transaction requests with a **lock-x**(Q) instruction.
- A data item Q is unlocked with an **unlock**(Q) instruction.
- A transaction must make a **lock request** to the **concurrency-control manager** before accessing a data item.
 - A transaction can proceed only after a request is granted.
 - A transaction should not request a lock until it is needed.

👉 **The concurrency control manager should allow *only* conflict-serializable schedules that are recoverable.**

LOCK-COMPATIBILITY MATRIX

		Requested	
		Shared (S)	Exclusive (X)
Held	Lock Mode		
Held	Shared (S)	grant	deny
	Exclusive (X)	deny	deny

Shared - can *only* read; Exclusive - can read and write

Locking Etiquette

To maximize concurrency, a transaction normally should request a *lock-x* only when it wants to write the data item.

- A transaction may be granted a lock on a data item *Q* if the requested *lock is compatible with locks already held* on *Q* by other transactions.
- *Any number of transactions* can hold *shared locks* on a data item *Q*.
- If any transaction holds an *exclusive lock* on a data item, then *no other transaction may hold any lock on the data item*.
- If a lock cannot be granted, *the requesting transaction is made to wait* until all incompatible locks held by other transactions have been *released*. *The lock is then granted*.

LOCKING PROTOCOL

Start with A=100, B=200		
	T_1	T_2
	read(B)	
	B := B - 50	
B=150	write(B)	
	read(A)	
	A := A - 50	
A=50	write(A)	
		read(A)
		read(B)
		display(A+B)
		200

To ensure schedule correctness, locking requires a set of rules, called a locking protocol, that all transactions follow when requesting and releasing locks. The rules restrict the set of possible schedules.

A possible schedule

T_1	T_2	CC Manager
lock-x(B)		grant-X(B)
read(B)		
B := B - 50		
write(B)		
unlock(B)		
	lock-s(A)	grant-S(A)
A=100	read(A)	
	unlock(A)	
	lock-s(B)	grant-S(B)
B=150	read(B)	
	unlock(B)	
	display(A+B)	250 INCORRECT!
lock-x(A)		grant-X(A)
read(A)		
A := A - 50		
write(A)		
unlock(A)		

Serializability may not be ensured if a transaction unlocks a data item immediately after its final access of that data item but before it commits.

TWO-PHASE LOCKING (2PL) PROTOCOL

Phase 1: Growing Phase (request or upgrade locks)

- Acquire a **lock-s** or **lock-x** or **upgrade** a **lock-s** to a **lock-x** on a data item.

Phase 2: Shrinking Phase (release or downgrade locks)

- Release a **lock-s** or **lock-x** or **downgrade** a **lock-x** to a **lock-s** on a data item.

Using 2PL, transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its **final lock**).

✎ If a schedule is **executed by 2PL**, then
it must be **conflict serializable**.

✎ If a schedule is **conflict serializable**, then
it may or may not be **executed by 2PL**.

✎ **Not all conflict serializable schedules are allowed by 2PL.**

Revised Locking Etiquette

To maximize concurrency, a transaction normally should request only a **lock-s** before reading a data item and upgrade it to a **lock-x** only when it wants to write the data item.

2PL AND RECOVERABILITY

Under (basic) 2PL, a schedule may not be recoverable and cascading roll-back is possible.

Example

- This schedule is allowed by basic 2PL since there is no cycle in the precedence graph.
- If T_1 aborts, then T_2 should be rolled back because it read data item A written by T_1 .
- However, since T_2 has committed it cannot be rolled back and so **the schedule is not recoverable**.

T_1	T_2
lock-x(A)	
write(A)	
unlock(A)	
	lock-s(A)
	read(A)
	commit
abort	

STRICT AND RIGOROUS TWO-PHASE LOCKING

Strict Two-phase Locking

- Requires that all exclusive-mode locks be held until after a transaction commits (shared-mode locks can be released anytime).

Rigorous Two-phase Locking

- Requires that all locks (both shared-mode and exclusive-mode) be held until after a transaction commits.
- Both protocols ensure that any data written by an uncommitted transaction are locked in exclusive mode until the transaction commits and so cannot be read by another transaction.
- The transactions can be serialized in the order in which they commit.

 **Strict and rigorous 2PL schedules are recoverable and cascadeless.**

LOCKING PROTOCOL ISSUE: STARVATION

Starvation (waiting forever) is possible if the concurrency control manager is **badly designed**.

- A transaction may be waiting for a **lock-x** on an item, while a sequence of other transactions request, and are granted, a **lock-s** on the same item.

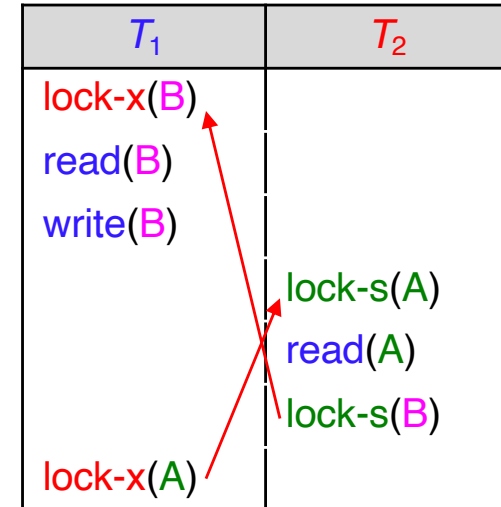
T_1	T_2	T_3	T_4	T_5
lock-s(A)	lock-x(A)			
	wait	lock-s(A)	lock-s(A)	lock-s(A)
unlock(A)				

👉 **A locking protocol needs to be able to handle starvation.**

LOCKING PROTOCOL ISSUE: DEADLOCKS

- A system is **deadlocked** if there is a set of transactions such that **every transaction is waiting for another transaction in the set**.
- Neither T_1 nor T_2 can make progress — executing **lock-s(B)** causes T_2 to wait for T_1 to release its lock on **B**, while executing **lock-x(A)** causes T_1 to wait for T_2 to release its lock on **A**.

T_1	T_2
lock-x(B) read(B) write(B)	
	lock-s(A) read(A) lock-s(B)
lock-x(A)	



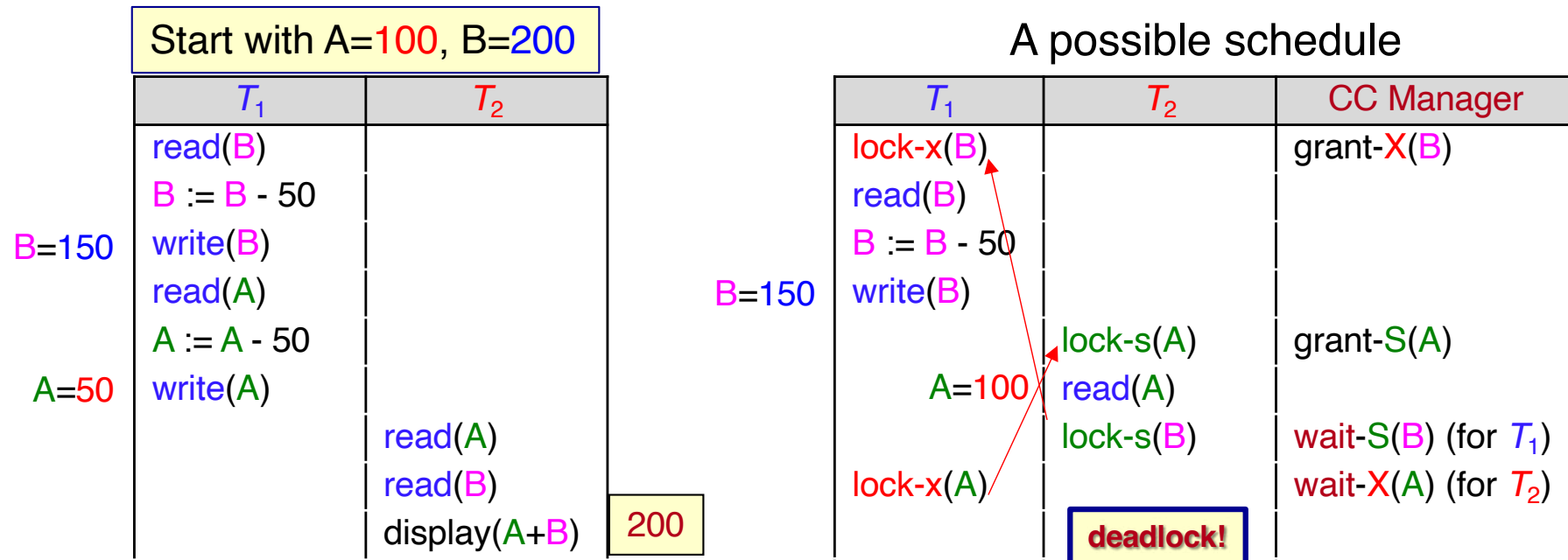
☞ Such a situation is called a **deadlock**.

☞ To handle a deadlock, either T_1 or T_2 must be **rolled back (aborted)** and its **locks released**.

☞ A rollback requires all a transaction's instructions to be **undone**.

☞ A **locking protocol** needs to be able to **handle deadlocks**.

DEADLOCK HANDLING



👉 2PL permits deadlocks.

The *potential* for deadlock exists in most locking protocols.
Deadlocks are a *necessary evil* of most locking protocols.

👉 A lock-based protocol can deal with the potential for deadlocks by either preventing them or detecting them.

DEADLOCK PREVENTION

Order lock requests

Impose a partial/total ordering of all data items and require transactions to *lock data items only in the order specified by the partial/total order*.

Pre-emptively rollback transactions *before* deadlock happens

A transaction waits for a lock only for a specified amount of time after which it is rolled back.

Wait-die Scheme

- An **older transaction** *waits* for a **younger one** to release a data item; a **younger transaction** never waits for an **older one** to release a data item but is rolled back (*dies*) instead.

Wound-wait Scheme

- A **younger transaction** *waits* for an **older one** to release a data item; an **older transaction** *wounds* (forces the rollback of) a **younger transaction** instead of waiting for it.

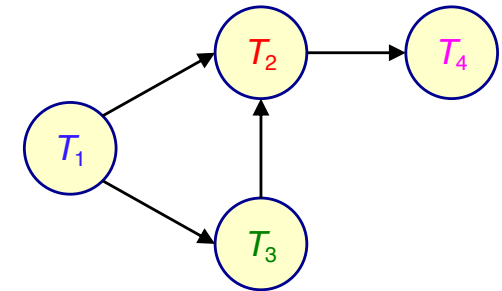
Starvation is possible; can be handled using timestamps as "age".

DEADLOCK DETECTION

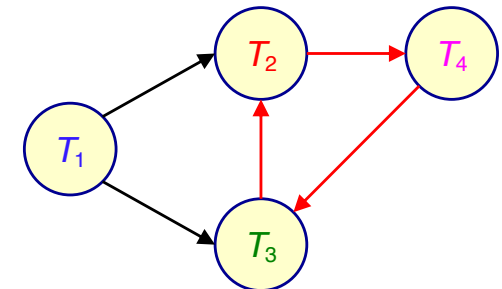
- Deadlocks can be detected using a **wait-for graph** $G = (V, E)$, where
 - V is a set of vertices representing all the transactions in the system.
 - E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.
 - If $T_i \rightarrow T_j$ is in E , then the directed edge from T_i to T_j implies that T_i is waiting for T_j to release a data item.
 - When T_i requests a data item currently being held by T_j , then the edge $T_i \rightarrow T_j$ is added to the wait-for graph.
 - This edge is removed only when T_j is no longer holding a data item needed by T_i .

✎ **The system is in a deadlock state if and only if the wait-for graph has a cycle.**

✎ **The system must invoke a deadlock-detection algorithm periodically to look for cycles in the wait-for graph.**



Wait-for graph without a cycle.



Wait-for graph with a cycle.

✎ **This is not the same as a precedence graph!**

DEADLOCK RECOVERY

Victim Selection

- Select as a **victim** the transaction that will **incur minimum cost**.

Rollback

- Need to determine how far to roll back the transaction.
 - **Total rollback**: abort the transaction and then restart it.
 - **Partial rollback**: roll back the transaction only as far as necessary to break the deadlock. (Requires the system to maintain additional information about transactions so it can determine how far to roll back a transaction.)

Starvation

- Can happen if the same transaction is always chosen as the victim.
- Can include the number of rollbacks in the victim selection process to avoid starvation.

IMPLEMENTATION OF LOCKING

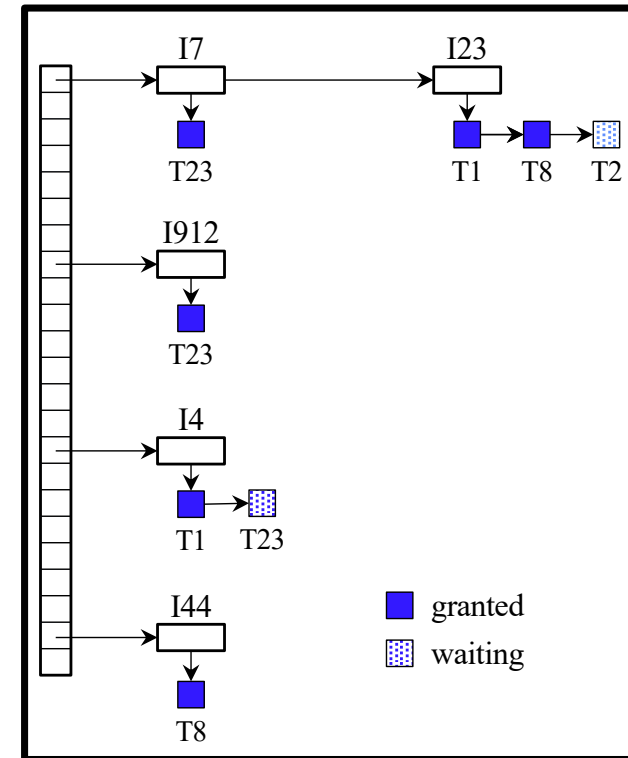
- A **lock manager** can be implemented as a separate process to which transactions send lock and unlock requests.

👉 **Transactions should not request locks until they are needed.**

- The lock manager replies to a lock request by sending a **lock grant message** (or a message asking the transaction to roll back in case of a deadlock).
- The requesting transaction **waits** until its request is answered.
- The lock manager maintains a data structure called a **lock table** to record granted locks and pending requests.
- The lock table is usually implemented as an **in-memory hash table** indexed on the name of the data item being locked.

IMPLEMENTATION OF LOCKING: LOCK TABLE

- Dark blue rectangles indicate **granted locks**, light blue ones indicate **waiting requests**.
 - The lock table also records the **type of lock** granted or requested.
 - A new request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks.
 - Unlock requests result in the request being deleted and waiting requests being checked to see if they can now be granted.
 - If a transaction aborts, all waiting or granted requests of the transaction are deleted.
- The lock manager also may keep a list of locks held by each transaction, to implement this efficiently.



➡ **This algorithm guarantees freedom from starvation. Why?**



TIMESTAMPS

Each transaction is issued a fixed **timestamp** $TS(T_i)$ by the system before it starts execution.

If the **most recent transaction** T_i has timestamp $TS(T_i)$, a **new transaction** T_j is assigned timestamp $TS(T_j)$ such that $TS(T_j) > TS(T_i)$.

👉 **The timestamps determine the serializability order.**

👉 If $TS(T_i) < TS(T_j)$, then T_i must not be allowed to see any results of T_j since T_j started later in time than T_i .

- To implement serializability, **two timestamp values** are associated with **each data item** Q .

RTS(Q) (**read timestamp**) the largest timestamp of any transaction that executed **read**(Q) successfully.

WTS(Q) (**write timestamp**) the largest timestamp of any transaction that executed **write**(Q) successfully.

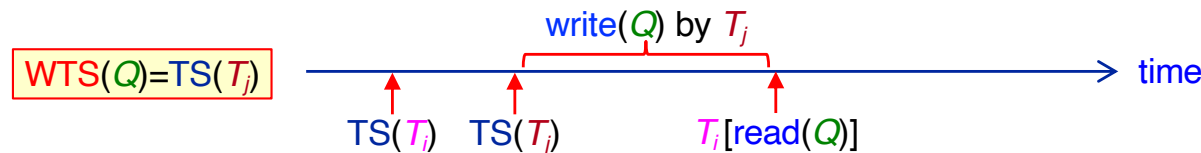
The timestamps of Q are updated whenever a new **read**(Q) or **write**(Q) instruction is executed.

TIMESTAMP-ORDERING PROTOCOL: READ

The timestamp-ordering protocol ensures that any **conflicting read** and **write** instructions are **executed in timestamp order**.

Suppose a transaction T_i issues a **read**(Q).

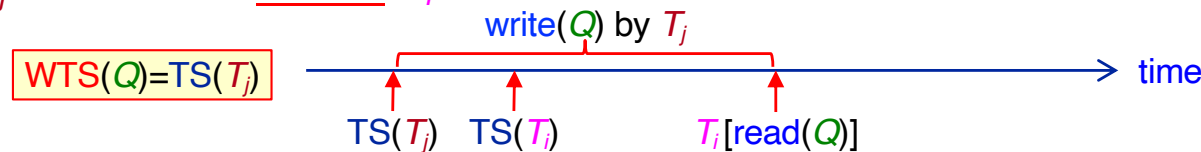
1. If $TS(T_i) < WTS(Q) = TS(T_j)$, then T_i is trying to read a value of Q that was already overwritten by a newer transaction T_j that started after T_i (since $TS(T_j) > TS(T_i)$).



✎ **Reject** the **read** instruction and **roll back** T_i .

✎ **Restart** T_i with a new (larger) timestamp $TS'(T_i)$.

2. If $TS(T_i) \geq WTS(Q) = TS(T_j)$, then execute the **read** instruction and set $RTS(Q)$ to the maximum of $RTS(Q)$ and $TS(T_i)$ since Q was written by an older transaction T_j that started before T_i .

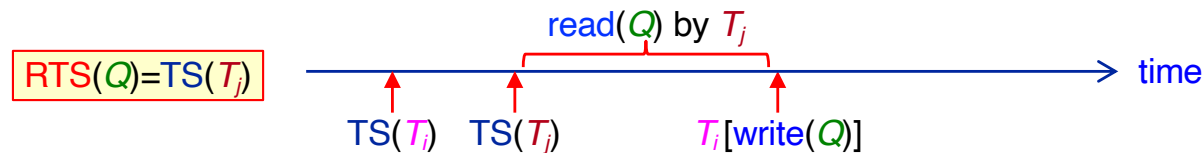


✎ T_i can read a data item Q iff $TS(T_i) \geq WTS(Q)$.

TIMESTAMP-ORDERING PROTOCOL: **WRITE**

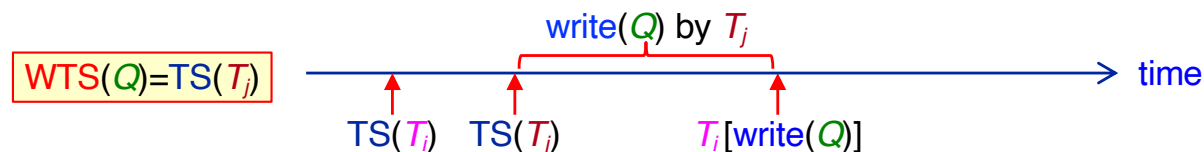
Suppose that transaction T_i issues a **write**(Q).

1. If $TS(T_i) < RTS(Q) = TS(T_j)$, then the value of Q that T_i is writing was **needed previously** and the system assumed it would never be written (since a newer transaction T_j that started **after** T_i already read a different value).



👉 **Reject** the **write** instruction and **roll back** T_i .

2. If $TS(T_i) < WTS(Q) = TS(T_j)$, then T_i is attempting to write an **obsolete value** of Q since a newer transaction T_j that started **after** T_i wrote a newer value of Q .



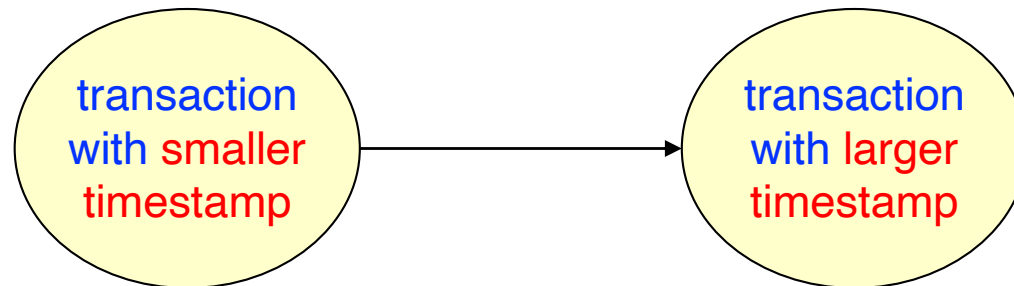
👉 **Reject** the **write** instruction and **roll back** T_i .

3. Otherwise, execute the **write** instruction and set $WTS(Q)$ to $TS(T_i)$.

👉 T_i can write a data item Q iff $TS(T_i) > RTS(Q)$ and $WTS(Q)$.

TIMESTAMP-ORDERING PROTOCOL CORRECTNESS

- The timestamp-ordering protocol ensures conflict serializability since all the arcs in the precedence graph are of the form:

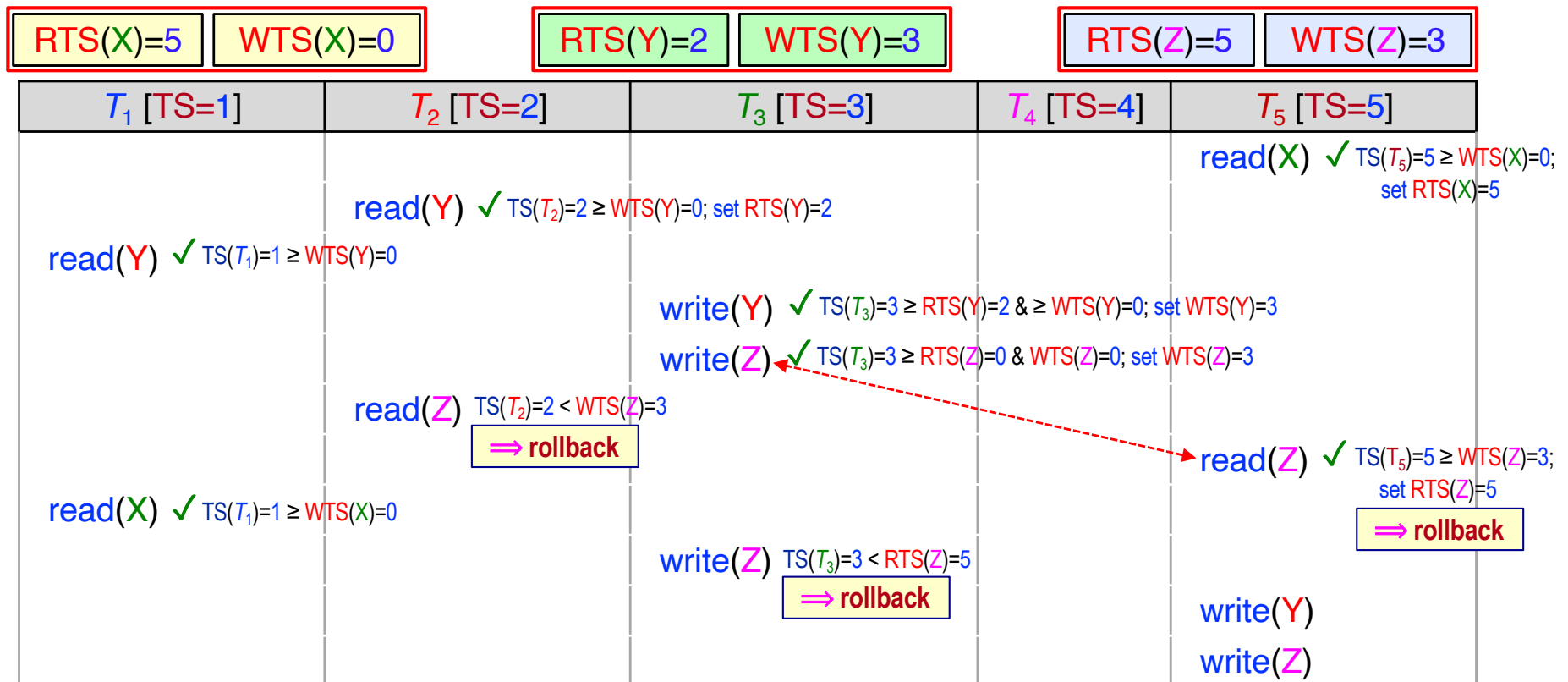


👉 Conflicting instructions are processed in timestamp order.

- Since there can be no cycles in the precedence graph, the timestamp-ordering protocol ensures freedom from deadlock and no transaction ever waits.
- However, starvation of long transactions is possible, and a schedule may not be recoverable. Why?

TIMESTAMP-ORDERING PROTOCOL EXAMPLE

Below is a partial schedule for several data items for transactions with timestamps 1, 2, 3, 4, 5. Assume initial R/W timestamp of all items is 0.



Read

If $TS(T_i) < WTS(Q)$ **rollback**
 If $TS(T_i) \geq WTS(Q)$
 $RTS(Q) = \max(TS(T_i), RTS(Q))$

Write

If $TS(T_i) < RTS(Q)$ **rollback**
 If $TS(T_i) < WTS(Q)$ **rollback**
 Otherwise $WTS(Q) = TS(T_i)$

RECOVERABILITY AND CASCADING ROLLBACK

- The problem with the timestamp-ordering protocol:
 - Suppose T_i aborts, but T_j has read a data item written by T_i .
 - Then T_j must abort; if T_j had been allowed to commit before T_i aborted, the schedule is not recoverable.
 - Moreover, any transaction that has read a data item written by T_j must abort.

 This can lead to cascading rollback.

Solution

- A transaction is structured such that *all its writes* are performed at the end of its processing.
- All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written.
- A transaction that aborts is restarted with a new timestamp.

THOMAS' WRITE RULE

Read

If $TS(T_i) < WTS(Q)$ **rollback**
 If $TS(T_i) \geq WTS(Q)$
 $RTS(Q) = \max(TS(T_i), RTS(Q))$

Write

If $TS(T_i) < RTS(Q)$ **rollback**
 If $TS(T_i) < WTS(Q)$ **rollback**
 Otherwise $WTS(Q) = TS(T_i)$

$RTS(Q)=1$ $WTS(Q)=2$

T_1 [TS=1]	T_2 [TS=2]
read(Q) ✓	$TS(T_1)=1 \geq WTS(Q)=0$; set $RTS(Q)=1$
	write(Q) ✓ $TS(T_2)=2 \geq RTS(Q)=1$ & $\geq WTS(Q)=0$; set $WTS(Q)=2$
write(Q)	$TS(T_1)=1 < WTS(Q)=2$ → ignore

👉 Rollback of T_1 is unnecessary!

- Since T_2 has already written Q (with $TS=2$), the value T_1 is attempting to write (with $TS=1$) *will never be read*.
 - Any transaction with timestamp less than $TS(T_2)$ that attempts to read Q will be rolled back since its timestamp will be less than $WTS(Q)$.
 - Any transaction with a timestamp greater than $TS(T_2)$ must read the value of Q written by T_2 , rather than the one T_1 is attempting to write.

👉 Ignore write of T_1 !

Read (unchanged)

If $TS(T_i) < WTS(Q)$ **rollback**
 If $TS(T_i) \geq WTS(Q)$
 $RTS(Q) = \max(TS(T_i), RTS(Q))$

Write (Thomas' write rule)

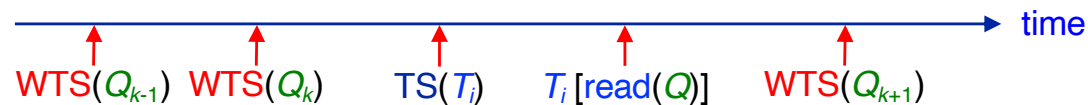
If $TS(T_i) < RTS(Q)$ **rollback**
 If $TS(T_i) < WTS(Q)$ **ignore**
 Otherwise $WTS(Q) = TS(T_i)$

MULTIVERSION SCHEMES

- Multiversion schemes **keep old versions of data items**, **labelled with timestamps**, to increase concurrency.
 - Multiversion Two-Phase Locking (2PL)
 - Multiversion Timestamp Ordering
- Each data item Q has a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$.
- Each version Q_k contains three data fields:
 - Content** The value of version Q_k .
 - WTS**(Q_k) The timestamp of the transaction that created (wrote) version Q_k .
 - RTS**(Q_k) The **largest timestamp** of a transaction that **successfully read** version Q_k .
- Each successful write by a transaction T_i creates a **new version** Q_k of Q and sets Q_k 's **RTS** and **WTS** to **TS**(T_i).

MULTIVERSION TIMESTAMP ORDERING: READ

Q_k is the version of Q whose write timestamp is the largest write timestamp *less than or equal to* $TS(T_i)$.



- If transaction T_i issues a $read(Q)$, then the value returned is the content of version Q_k (i.e., T_i reads the **most recent version** that comes before it in time).

👉 If $TS(T_i) > RTS(Q_k)$, then set $RTS(Q_k) = TS(T_i)$.

👉 **Reads never fail and never wait** as an appropriate version can always be found.

Read

Reads always succeed

set $RTS(Q_k) = \max(TS(T_i), RTS(Q_k))$

MULTIVERSION TIMESTAMP ORDERING: **WRITE**

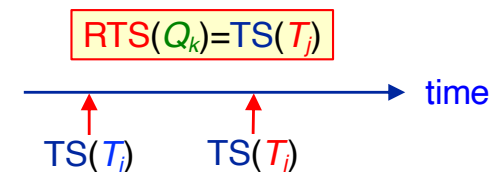
Q_k is the version of Q whose write timestamp is the largest write timestamp *less than or equal to* $TS(T_i)$.

If transaction T_i issues a **write**(Q), then

- a) if $TS(T_i) < RTS(Q_k) = TS(T_j)$, then **roll back** T_i .

Since a transaction T_j , whose timestamp is larger than that of T_i and so should have read the version of Q that T_i is trying to write (in the serialization order defined by the timestamp values), instead read a version of Q created by a transaction older than T_i .

- b) if $TS(T_i) = WTS(Q_k)$, **overwrite** the **contents** of $Q_k \Rightarrow Q_k$ was written before also by T_i .
- c) if $TS(T_i) > WTS(Q_k)$ **create** a **new version** of Q and set **R/WTS** of new version to $TS(T_i)$.



✎ **Conflicts are resolved through aborting transactions.**

Write

If $TS(T_i) < RTS(Q_k)$ **rollback**

If $TS(T_i) = WTS(Q_k)$ **overwrite contents**

If $TS(T_i) > WTS(Q_k)$ **create new version**

set $R/WTS(Q') = TS(T_i)$

MULTIVERSION SCHEMES: NOTES

- The schedule of **Exercise 3** will terminate successfully although it is **not conflict serializable**.
- The concept of **conflict serializability** does not apply to multi-version protocols, since **read instructions do not conflict with previous writes** (since there is always a version to read).

Versions are removed as follows:

- Suppose that there are two versions Z_1 and Z_2 with timestamps **3** and **1** (as could happen in **Exercise 3**) that both have a smaller timestamp than the oldest transactions (i.e., T_1 , T_2 and T_3 have committed, and the new transactions have timestamps ≥ 4).
- Then the version Z_2 with timestamp **1** will not be used again and is deleted.
- If any subsequent transaction needs to read Z , it will read Z_1 with timestamp **3**.
- In **Exercise 3** it is as if the last write(Z) was never executed, which is OK because T_3 executes a **blind** write(Z) (i.e., write with no previous read).
- If, on the other hand, T_3 had read Z before writing it, the schedule would fail at the last write of T_1 .